University of Eswatini

Department of Computer Science

Part 2: System Design

Integrated REST-based System for Justice Administration

(Police, Judiciary and Correctional Services)

Course	CSC411 Integrative Programming Technologies
Assessment	Assignment 1 - Mini Project (2026)
Instructor	Dr MA Nxumalo
Prepared by	Melody Sylvester (202303641)
Team members	Thwala Phiwayini - 202403175 Motsa Kwanele - 155307 Thandinkosi Magagula - 202302756 Melody Sylvester - 202303641 Sibusiso Maseko - 202301241
Submission date	4 November 2026, 23h00

1. Introduction and Scope

This document presents the design of an integrated, REST-based system that automates the **administration of justice** in Eswatini. The process involves three independent organisations: the **Police**, who register a suspect, record an arrest, open a case and add charges; the **Judiciary**, who receive the case, schedule hearings, decide on bail or remand, and deliver a verdict and sentence; and the **Correctional Services**, who admit, manage, transfer and release inmates.

Today these organisations exchange paper files and phone calls, which causes delays, lost records and inconsistent information. The aim of this design is to let the three organisations keep full control of their own systems and data while exchanging information automatically through a shared, well-defined REST API contract.

1.1 Design goals

- **Autonomy:** each organisation owns its service and its database.
- **Interoperability:** all services follow one OpenAPI contract so that four developers can build in parallel.
- **Traceability:** every case can be followed from arrest to release using shared reference IDs.
- **Security:** only authenticated and authorised users and services may call the APIs.
- **Simplicity:** a technology stack the team can implement and demonstrate within the project period.

1.2 Stakeholders and roles used in the design

Role (JWT claim)	Organisation	Main responsibilities
police_officer	Police	Register suspects, record arrests, create cases, add charges, refer cases
court_officer	Judiciary	Receive cases, schedule hearings, record bail, remand, verdict and sentence
correctional_officer	Correctional Services	Admit inmates, view status, transfer and release
admin	All	Manage users, view all records (for demonstration and testing)

1.3 The Eswatini context

The three services represent real Eswatini institutions: the **Royal Eswatini Police Service** (Police), the **Judiciary of Eswatini** (Magistrates Courts and the High Court, with the Director of Public Prosecutions presenting cases) and the **Correctional Services** (correctional centres such as Matsapha). Bail and trial procedure are mainly governed by the **Criminal Procedure and Evidence Act 67 of 1938** (as amended), where section 96 deals with bail, and by the **Constitution of the Kingdom of Eswatini Act 2005**, which protects personal liberty and a fair hearing. The design therefore models bail and remand as separate court decisions, requires a court decision before any person is admitted to custody or released by the Correctional Services, and keeps an audit trail. This is a proposal for academic purposes.

2. Functional Requirements

Functional requirements describe what the system must do.

ID	Service	Requirement
FR-01	All	The system shall authenticate users with a username and password and issue a JWT containing the user's role.
FR-02	Police	The system shall allow a police officer to register a suspect with name, national ID, date of birth and address.
FR-03	Police	The system shall record an arrest (date, location, arresting officer) for a registered suspect.
FR-04	Police	The system shall create a case from an arrest and assign a unique case reference.
FR-05	Police	The system shall allow one or more charges (offence and statute) to be added to an open case.
FR-06	Police	The system shall refer a case with at least one charge to the Judiciary service.
FR-07	Judiciary	The system shall receive a referred case and store the suspect and charges it contains.
FR-08	Judiciary	The system shall schedule hearings for a case (date, court, presiding officer).
FR-09	Judiciary	The system shall record a bail decision with an amount and conditions.
FR-10	Judiciary	The system shall record a remand decision and request admission of the suspect at a correctional facility.
FR-11	Judiciary	The system shall record a verdict (guilty or not guilty) and, when guilty, a sentence.
FR-12	Judiciary	The system shall inform the Correctional service when a custodial sentence is imposed: for a suspect already on remand it updates the inmate to CONVICTED with the sentence term (PUT /inmates/{id}/status); otherwise it admits a new inmate as CONVICTED (POST /inmates).
FR-13	Correctional	The system shall admit an inmate as REMAND or CONVICTED and assign an inmate ID.
FR-14	Correctional	The system shall return the current status and details of an inmate.
FR-15	Correctional	The system shall transfer an inmate to another facility and keep the history.
FR-16	Correctional	The system shall release an inmate and record the reason and date.
FR-17	Frontend	The web frontend shall let each role perform its operations and view the case status end to end.
FR-18	All	The system shall write an audit log entry (time, user, role, action, record ID) for every create and update; admins can read it.
FR-19	All	The system shall let authorised users look up the latest status of a case or inmate by its reference ID, so officers in other organisations can see changes.
FR-20	Judiciary	The system shall let the court order the release of a remanded suspect (not guilty verdict, or bail granted); the Judiciary sends the release order to the Correctional service (POST /inmates/{id}/release) and the inmate is recorded as RELEASED.

3. Non-Functional Requirements

Non-functional requirements describe how well the system must work.

ID	Category	Requirement
NFR-01	Security	All endpoints except login shall require a valid JWT; roles shall be enforced on every operation. Passwords shall be stored hashed (bcrypt).

ID	Category	Requirement
NFR-02	Performance	Single-record operations shall respond in under 2 seconds on a normal laptop, including service-to-service calls.
NFR-03	Reliability	If a downstream service is unavailable, the caller shall return a clear error and shall not leave a half-completed update.
NFR-04	Data integrity	Each service shall validate input and enforce valid state changes (for example, a case cannot be referred twice).
NFR-05	Interoperability	All services shall follow the shared OpenAPI contract, use JSON and standard HTTP status codes.
NFR-06	Maintainability	Each service shall be a separate project with its own README, so it can be changed without touching the others.
NFR-07	Scalability	Services shall be stateless so that more instances could be run behind a load balancer in future.
NFR-08	Auditability	Every create and update shall store a timestamp and the user who performed it.
NFR-09	Usability	The frontend shall work in a standard browser and show clear messages for errors.
NFR-10	Portability	The system shall run locally with a single command per service and no paid software.

4. System Architecture

The system uses a **service-oriented architecture with three services**, one for each organisation. Each service exposes a REST API over HTTP, owns its own database, and is deployed independently on its own port. Figure 1 shows the structure.

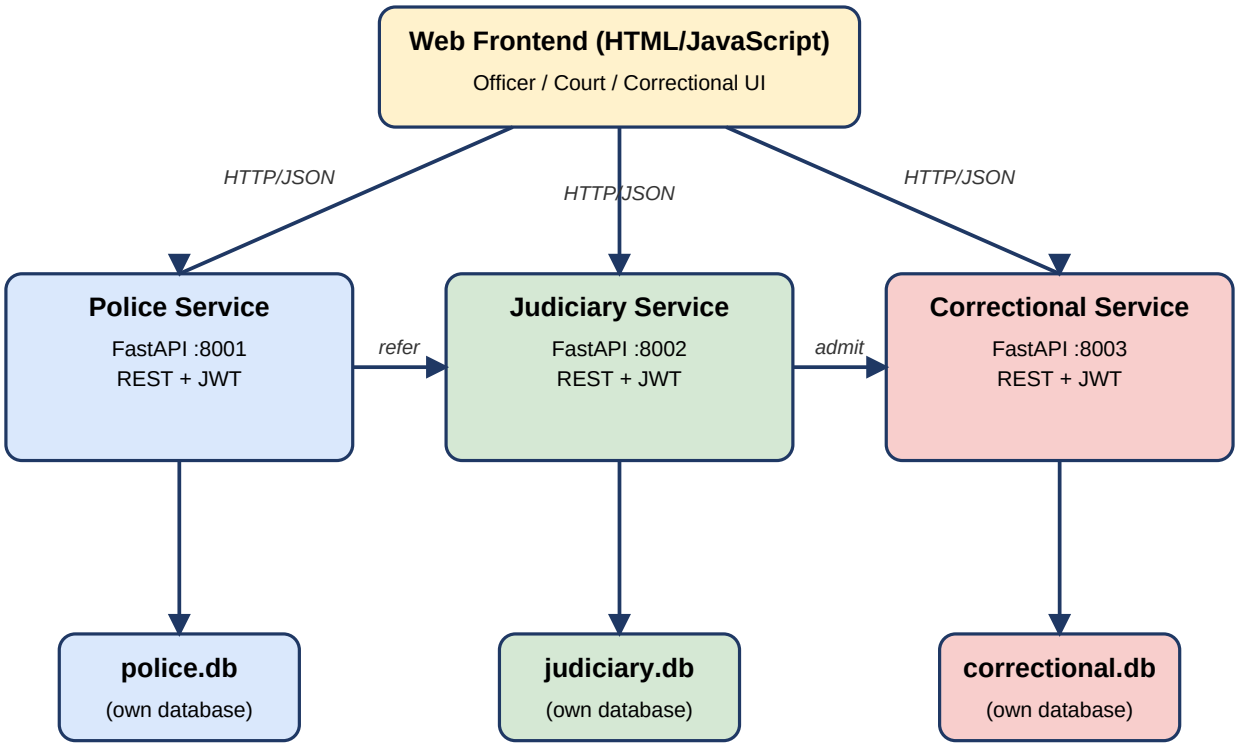


Figure 1: Three-service architecture with one database per service

4.1 The three services

Service	Port	Owner	Database	Owns (source of truth)
Police Service	8001	Police	police.db	Suspects, arrests, police cases, charges
Judiciary Service	8002	Judiciary	judiciary.db	Court cases, hearings, bail, remand, verdicts, sentences
Correctional Service	8003	Correctional Services	correctional.db	Inmates, admissions, transfers, releases

4.2 Database per service

Each service has its own private database and no service reads another service's database directly. This is the *database-per-service* pattern. It matches the real world (the Police and the Courts are separate institutions with separate records) and it keeps the services loosely coupled: a change to the Police tables cannot break the Judiciary service. When one service needs data owned by another, it asks through the API, or receives a copy in the request body. For example, when a case is referred, the Police service sends the suspect and charge details inside the request, and the Judiciary stores its own copy together with the police_case_id for traceability.

Because data is copied across services, the design accepts **eventual consistency** for copied fields, while the status fields (case status, inmate status) are always owned by exactly one service.

4.3 Layers inside each service

- **Router/controller layer:** defines endpoints, validates input and checks the JWT role.
- **Service layer:** business rules (for example, valid status changes) and calls to other services.
- **Repository layer:** reads and writes the service's own database.
- **Model/schema layer:** Pydantic schemas for requests and responses, matching the OpenAPI file.

Layering keeps the code easy to read and test, and every member follows the same internal structure.

4.4 Class diagram

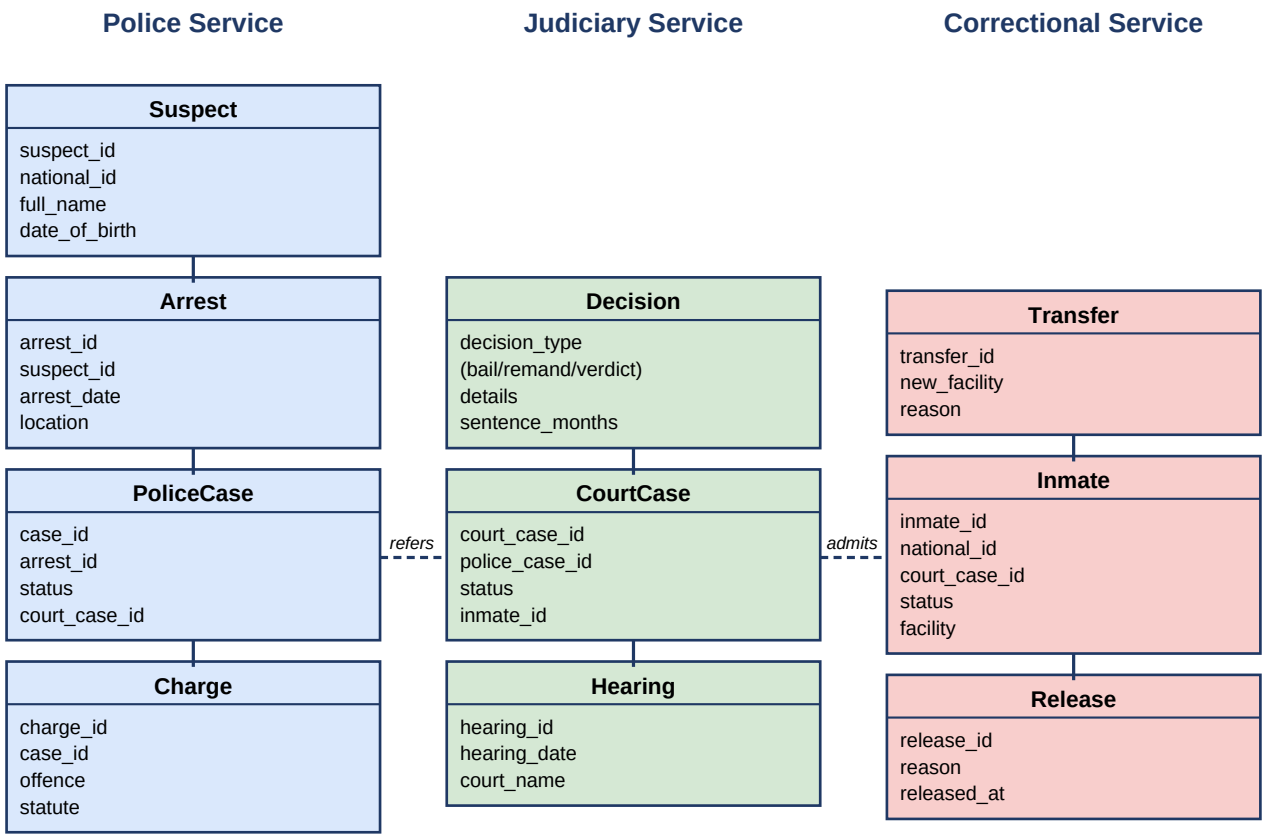


Figure 2: Class diagram. Solid lines are relationships inside a service (one-to-many, top to bottom); dashed lines are references between services by ID only.

5. Component Communication and Workflow

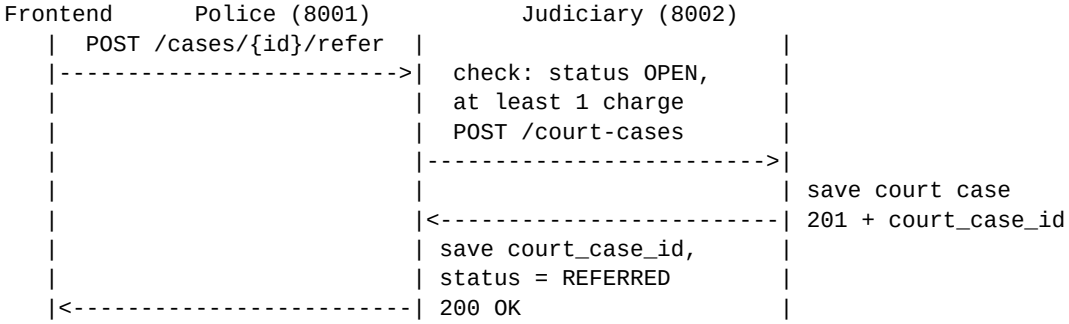
The frontend calls the services directly using HTTP and JSON. Services call each other only where the process requires it: Police to Judiciary when a case is referred, and Judiciary to Correctional when a suspect is remanded or sentenced to imprisonment. Correctional never calls back, so there are no circular dependencies.

5.1 End-to-end process flow

Step	Actor	API call	Result
1	Police officer	POST /auth/login (8001)	JWT issued
2	Police officer	POST /suspects	Suspect SUS-xxxx created
3	Police officer	POST /arrests	Arrest ARR-xxxx recorded
4	Police officer	POST /cases, POST /cases/{id}/charges	Case OPEN with charges
5	Police officer	POST /cases/{id}/refer	Police calls Judiciary POST /court-cases; case becomes REFERRED
6	Court officer	POST /court-cases/{id}/hearings	Hearing scheduled
7a	Court officer	POST /court-cases/{id}/bail	Status ON_BAIL. If the suspect was on remand, Judiciary calls Correctional POST /inmates/{id}/release with reason BAIL_GRANTED

Step	Actor	API call	Result
7b	Court officer	POST /court-cases/{id}/remand	Judiciary calls Correctional POST /inmates (REMAND); status ON_REMAND
8a	Court officer	POST /court-cases/{id}/verdict (GUILTY, IMPRISONMENT)	Suspect on remand: Judiciary calls PUT /inmates/{inmate_id}/status with status CONVICTED and sentence_months. Not on remand: Judiciary calls POST /inmates with admission_type CONVICTED and sentence_months
8b	Court officer	POST /court-cases/{id}/verdict (NOT_GUILTY)	Suspect on remand: Judiciary calls POST /inmates/{inmate_id}/release with reason ACQUITTED. Case becomes CLOSED
9	Correctional officer	GET /inmates/{id}, PUT /inmates/{id}/transfer	Inmate status viewed; transfer recorded
10	Correctional officer	POST /inmates/{id}/release	Inmate RELEASED (sentence completed; releases ordered by the court come from steps 7a and 8b)

5.2 Sequence for referring a case (service-to-service)



5.3 Case status life cycle

Owner	States and allowed transitions
Police case	OPEN -> REFERRED -> CLOSED
Court case	RECEIVED -> HEARING_SCHEDULED -> ON_BAIL or ON_REMAND -> SENTENCED or CLOSED
Inmate	REMAND -> CONVICTED -> TRANSFERRED (optional) -> RELEASED

Invalid transitions (for example, granting bail on a closed case) are rejected with HTTP 409 Conflict. Keeping the life cycle inside the owning service guarantees the rules are applied in one place only.

5.4 Which call the Judiciary makes after each court decision

After a decision, the Judiciary makes exactly one call to the Correctional service (or none), as follows. The Judiciary stores the inmate_id returned by the first admission, so it always knows whether a suspect is on remand.

Court decision	Suspect on remand? (inmate_id stored)	Call to Correctional service	Data carried	Result
Remand	No	POST /inmates	admission_type REMAND, facility, national_id, court_case_id	Inmate REMAND; inmate_id saved
Bail	Yes	POST /inmates/{id}/release	reason BAIL_GRANTED	Inmate RELEASED
Bail	No	none	-	Case ON_BAIL
Guilty, imprisonment	Yes	PUT /inmates/{id}/status	status CONVICTED, sentence_months	Inmate CONVICTED with sentence term
Guilty, imprisonment	No	POST /inmates	admission_type CONVICTED, sentence_months, facility	New inmate CONVICTED
Guilty, fine or suspended	Any	none (release first if on remand)	reason COURT_ORDER	Case CLOSED
Not guilty	Yes	POST /inmates/{id}/release	reason ACQUITTED	Inmate RELEASED; case CLOSED
Not guilty	No	none	-	Case CLOSED

If the call to the Correctional service fails, the Judiciary returns 502 and does **not** save the decision, so the two systems never disagree. A second admission for the same court case returns 409.

6. Security Design (JWT)

Security is based on **JSON Web Tokens (JWT)**. A user logs in with a username and password at POST /auth/login and receives a signed token. The token is then sent in the header Authorization: Bearer <token> with every request.

6.1 Token content

```
{
  "sub": "officer1",
  "role": "police_officer",
  "iat": 1790000000,
  "exp": 1790003600
}
```

6.2 Rules

- Tokens are signed with **HS256** using a secret key shared by the three services through an environment variable. This is acceptable for a demonstration; a production system would use RS256 public/private keys or a central identity provider.
- Tokens expire after 60 minutes. Expired or tampered tokens return **401 Unauthorized**.

- Each endpoint declares the roles allowed. A valid token with the wrong role returns **403 Forbidden**.
- For service-to-service calls, the calling service forwards the user's token, so the receiving service applies the same checks and the audit trail shows the original user. The Correctional service therefore accepts the role **court_officer** only on three operations: POST /inmates, PUT /inmates/{id}/status and POST /inmates/{id}/release.
- Passwords are hashed with bcrypt and never returned by any endpoint.
- Secrets are kept in environment variables and are never committed to GitHub.
- **Audit logging:** every create, update and referral writes a row to the service's audit_log table (time, username, role, action, record ID). Admins can read it with GET /audit-logs.

Endpoint group	police_officer	court_officer	correctional_officer	admin
Police: suspects, arrests, cases, charges, refer	Yes	Read only	No	Yes
Judiciary: court cases, hearings, bail, remand, verdict	No (service call only for receive)	Yes	Read only	Yes
Correctional: admit, status, transfer, release	Read only	Admit, update status and release, only through Judiciary service calls	Yes	Yes

7. Design Patterns

The following patterns are used. Each one solves a specific problem in this system.

Pattern	Where it is used	Why
Microservice / service-oriented	Three separate services	Matches three separate organisations; independent development and deployment.
Database per service	Each service has its own database	Loose coupling and data ownership.
REST resource-oriented API	All endpoints (nouns such as /cases, /inmates)	Standard, simple and easy to test with Postman.
Layered architecture	Router, service, repository, model inside each service	Separation of concerns; easier testing.
Repository pattern	Database access	The business code does not depend on the database engine.
DTO (data transfer object)	Pydantic request/response schemas	Controls exactly what data crosses the API boundary.
API contract first	openapi.yaml written before coding	All developers build to the same interface.
Token-based authentication	JWT on every request	Stateless security that works across services.
State machine	Case and inmate life cycles	Prevents invalid transitions and keeps rules in one place.

8. Technology Stack and Justification

The assignment allows Python with Flask or FastAPI, Java with Spring Boot, or Node.js with Express. The team plan is **Python 3.10+ with FastAPI**.

Layer	Choice	Justification
Back-end language	Python 3.10 or newer	Easy to learn and read; every team member can understand each other's service.
Web framework	FastAPI	Generates OpenAPI documentation automatically, validates input with Pydantic, supports JWT easily and is lightweight.
Database	SQLite (one file per service) with SQLAlchemy	No installation or cost, one database file per service makes the database-per-service rule obvious; easy to replace with PostgreSQL later.
Authentication	PyJWT and passlib (bcrypt)	Standard, well documented libraries.
Service-to-service calls	httpx (or requests)	Simple HTTP client with timeout support.
Front-end	HTML, CSS and JavaScript (fetch API)	Allowed by the assignment; no build tools needed.
API specification	OpenAPI 3.0 (YAML)	Industry standard; viewable in Swagger Editor and importable into Postman.
Testing	Postman collection	Required by the project plan; can run the whole flow as one collection.
Version control	Git and GitHub	Required by the assignment; supports pull requests and review.
API documentation	Swagger UI (built into FastAPI)	Opens at /docs on each service and shows the same contract as openapi.yaml.
Diagrams	draw.io or Mermaid	Free tools for the architecture and class diagrams.
Editor	Visual Studio Code	Free, with Python and REST client extensions.
Data format	JSON over HTTP/REST	Human readable and supported by every tool in the stack.

Alternatives considered. Node.js with Express would also work, but Python with FastAPI gives automatic validation and documentation, which reduces the amount of code to write. Spring Boot is powerful but heavier to set up. A message broker (such as RabbitMQ) was considered for asynchronous communication but rejected because the process is a simple request-response flow and a broker would add complexity without benefit for this project. This matches the technology plan agreed by the team.

9. API Contract (OpenAPI)

The file docs/api-contract/openapi.yaml is the single source of truth for the interfaces. The three services follow the paths, field names, status codes and error format written there.

9.1 Endpoint summary

Service	Method and path	Purpose	Success
All	POST /auth/login	Obtain JWT	200

Service	Method and path	Purpose	Success
All	GET /audit-logs	Read audit trail (admin only)	200
Police	POST /suspects	Register suspect	201
Police	GET /suspects/{id}	Get suspect	200
Police	POST /arrests	Record arrest	201
Police	POST /cases	Create case	201
Police	GET /cases/{id}	Get case with charges	200
Police	POST /cases/{id}/charges	Add charge	201
Police	POST /cases/{id}/refer	Refer case to court	200
Judiciary	POST /court-cases	Receive referred case	201
Judiciary	GET /court-cases/{id}	Get court case	200
Judiciary	POST /court-cases/{id}/hearings	Schedule hearing	201
Judiciary	POST /court-cases/{id}/bail	Grant bail	200
Judiciary	POST /court-cases/{id}/remand	Remand in custody	200
Judiciary	POST /court-cases/{id}/verdict	Verdict and sentence	200
Correctional	POST /inmates	Admit inmate	201
Correctional	GET /inmates/{id}	Inmate status	200
Correctional	PUT /inmates/{id}/status	Update status	200
Correctional	PUT /inmates/{id}/transfer	Transfer inmate	200
Correctional	POST /inmates/{id}/release	Release inmate (sentence done, or court release order)	200

9.2 Common rules

- **Identifiers:** text IDs such as SUS-0001, CASE-2026-0001, CRT-2026-0001, INM-0001.
- **Dates:** ISO 8601 (for example 2026-10-20T09:00:00).
- **Status codes:** 200, 201, 400 (validation), 401, 403, 404, 409 (wrong state), 502 (downstream failure).
- **Error body (all services):** {"error": "NOT_FOUND", "message": "Suspect SUS-0099 does not exist"}.
- **Example request** - POST /suspects:

```
{
  "full_name": "Sipho Dlamini",
  "national_id": "8001015800083",
  "date_of_birth": "1980-01-01",
  "gender": "male",
  "address": "Ngwane Park, Manzini"
}
```

10. Data Design per Service

The table below gives the main entities each database contains so that the API contract can be satisfied.

Service	Main tables	Key fields
Police	users, suspects, arrests, cases, charges, audit_log	suspect_id, national_id, arrest_id, case_id, status, court_case_id (nullable until referred)
Judiciary	users, court_cases, hearings, decisions (bail, remand, verdict), audit_log	court_case_id, police_case_id, status, hearing_date, verdict, sentence_months, inmate_id (nullable)
Correctional	users, inmates, transfers, releases, audit_log	inmate_id, national_id, court_case_id, status, facility, sentence_months, admitted_at, released_at

Linking records across services. No foreign keys cross service boundaries. Instead, services store *reference IDs* as plain text: the Judiciary stores police_case_id, the Police stores court_case_id, and the Correctional service stores court_case_id. The suspect is identified across all three services by the national_id. These reference IDs make it possible to trace one person's case from arrest to release.

11. Error Handling and Resilience

- **Timeouts:** service-to-service calls use a 5-second timeout so that a slow service does not freeze the caller.
- **Fail without partial updates:** the caller changes its own record only after the downstream call succeeds. For example, a remand is saved in the Judiciary database only after the Correctional service has admitted the inmate. If the call fails, the caller returns **502 Bad Gateway** and the user can retry.
- **Idempotency checks:** referring a case twice, or admitting the same court case twice, returns **409 Conflict** instead of creating duplicates.
- **Validation:** missing or wrongly formatted fields return **400** with a clear message.
- **Logging:** each service logs the request, user, result and any downstream call, which helps debugging during the demonstration.

Known limitation. The design does not include automatic retry queues or distributed transactions. For a production system these would be added (for example, a message queue with retries), but they are outside the scope of this mini project.

12. Conclusion of Part 2

This design splits the justice process into three autonomous REST services that cooperate through a shared OpenAPI contract. Database-per-service keeps each organisation in control of its own data, JWT provides consistent security, and clear state rules keep the process correct from arrest to release. The chosen technology stack (Python, FastAPI, SQLite, HTML/JavaScript, Postman and GitHub) is simple enough to implement within the project period while still demonstrating the key integration concepts. Part 3 implements these services exactly as specified in this document and in openapi.yaml.

References

- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.
- Richardson, C. (2018). Microservices Patterns. Manning Publications.

- Jones, M., Bradley, J. and Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, IETF.
- OpenAPI Initiative (2021). OpenAPI Specification v3.0.3. https://spec.openapis.org/oas/v3.0.3
- Ramirez, S. FastAPI documentation. https://fastapi.tiangolo.com
- Postman Inc. Postman Learning Center. https://learning.postman.com

Appendix A: The OpenAPI contract (openapi.yaml)

The complete file openapi.yaml (OpenAPI 3.0.3, 20 paths) is submitted with the source code and is stored in the GitHub repository at /docs/api-contract/openapi.yaml. Key parts are shown here: the verdict, status update and release operations that connect the Judiciary and Correctional services.

```
openapi: 3.0.3
info: {title: Eswatini Justice Administration Integration API, version: 1.0.0}
servers: [8001 Police, 8002 Judiciary, 8003 Correctional]
security: [bearerAuth]          # JWT on every call except /auth/login

/court-cases/{court_case_id}/verdict:        # Judiciary (8002)
  post:
    requestBody: VerdictDecision
      verdict:         GUILTY | NOT_GUILTY
      sentence_type:   IMPRISONMENT | FINE | SUSPENDED
      sentence_months: integer (required for IMPRISONMENT)
    effect:  on remand + guilty   -> PUT  /inmates/{id}/status (CONVICTED)
             not on remand+guilty -> POST /inmates (CONVICTED)
             on remand + not guilty -> POST /inmates/{id}/release (ACQUITTED)
    responses: 200 CourtCase | 400 | 502

/inmates/{inmate_id}/status:                 # Correctional (8003)
  put:
    requestBody: InmateStatusUpdate
      status:          REMAND | CONVICTED
      sentence_months: integer (required when CONVICTED)
    roles: correctional_officer, admin, court_officer (service call)
    responses: 200 Inmate | 404

/inmates/{inmate_id}/release:                # Correctional (8003)
  post:
    requestBody: ReleaseRequest
      reason: SENTENCE_COMPLETED | ACQUITTED | COURT_ORDER | BAIL_GRANTED
      notes:  string
    roles: correctional_officer, admin, court_officer (court release order)
    responses: 200 Inmate | 409 (already released)

Error body (all services):  {"error": "NOT_FOUND", "message": "..."}
```